`this` in Functions & Classes

Understanding `this` in arrow function (=>) and regular function (function)

`this` and Its Interpretation

Imagine you have a toy box.

When you say:

> “Show me **my** toy!”

someone needs to know who “me” is.

That’s what `this` means in JavaScript — it’s like saying “me”.

Imagine you have a toy box again.
When you say to arrow function (=>):

> “Show me **my** toy!”

But arrow functions don’t ask anything about "me" — arrow functions **borrow “me”** from where they (arrow functions) were born.

That’s what makes them special.

Arrow Function Example

```
const person = {
  name: "Alice",
  normal: function() { // regular function: `this` is decided when it is called
    console.log("Normal:", this.name);
  },
  arrow: () => { // arrow function: `this` comes from where it was created
    console.log("Arrow:", this.name);
  }
};

person.normal();
person.arrow();
```

What happens when you write this code and when this file runs:

1. JavaScript creates the object person (dynamically at runtime).
2. It fills it with:

- a property name: "Alice"
- a property normal containing a regular function
- a property arrow containing an arrow function

3. The person object is dynamically created in memory when the code runs.

- But that’s not the tricky part — the tricky part is how this is set when you call the functions.

For person.normal()

When you call: `person.normal();`

JavaScript looks at the left side of the dot (person) and says:

> “Okay, person is the one calling this function — so this = person.”

Therefore, inside normal(), `this.name = "Alice"`.

For person.arrow()

When you call: `person.arrow();`

Arrow functions don’t care who calls them!

- They keep their `this` from where they were created (the lexical scope).
- In this case, the arrow function was created when the script loaded, and the surrounding scope was the global (or module) scope — not the person object.

So:

- The arrow function’s `this` is the `this` of the module scope. (In a Node.js CommonJS file, it is `module.exports`: an empty object `{}`.)
- `this.name` → `undefined`

Analogy: Birth vs Adoption

Function type	“Who am I?” is decided by	Analogy
Regular function	Who calls me	Adoption: “I live with whoever called me.”
Arrow function	Where I was created (the lexical parent)	Birth: “I always remember where I was born.”

So even if person.arrow is stored inside person,
it remembers its birthplace — the outer (module) scope.

What Happens

```
person.normal(); // this.name?
person.arrow(); // this.name?
```

When you run this:

```
Normal: Alice
Arrow: undefined
```

Why?

- The normal function finds its parent (adoption): it is called through the `person` object, so `person.name` is returned.
- The arrow function knows where it was born (birth): the module scope, where `this` is `{}` (an empty object). There is no `name` in it, so `undefined` is returned.

Function Example

Without "use strict";

```
function showThis() {
  console.log(this);
}
const showThis2 = () => {
  console.log(this);
};
```

- `showThis` (adoption): nobody called it as a method, so in non-strict mode `this` is the global object (`globalThis` ; `window` in a web browser), and the whole global object is printed.
- `showThis2` (birth): its `this` is the `this` of the module scope, so it prints `{}` (an empty object).

With "use strict";

- `showThis` (adoption): strict mode does not allow `this` to default to the global object, so `undefined` is printed.
- `showThis2` (birth): its `this` is still the `this` of the module scope, so it prints `{}` (an empty object).

Inner function Example (Lexical Scoping)

Normal Function (function)

```
const person = {
  name: "Alice",
  greet: function() {
    console.log("Outer:", this.name); // "Alice"
    function inner() {
      console.log("Inner:", this.name); // ❌ undefined (global object, or undefined in strict mode)
    }
    inner();
  },
};
person.greet();
```

- Regular functions have their own `this`.
- When you call `inner()` directly, as in this example, nobody is the caller.

JavaScript says:

> “No one called me — I don’t know who I belong to!”

- In non-strict mode, `this` is the global object, which has no `name`, so `undefined` is printed.
- In strict mode, `this` is `undefined`, so `this.name` throws a TypeError!

Arrow Function (=>)

```
const person2 = {
  name: "Alice",
  greet: function() {
    console.log("Outer:", this.name); // "Alice"

    const inner = () => {
      console.log("Inner:", this.name); // ✅ "Alice"
    };

    inner();
  },
};
```

Arrow functions don’t have their own this.

- Instead, they borrow this from the place they were created: Inside person2.greet(), which is a regular function where this already points to person2.
- So the arrow function’s this = same as its parent (person2).

Database Example

In database, we sometimes need to use `this` for accessing information.

- In this case, we must use a regular function, not an arrow function.

```
// ❌ Arrow function — won't work with lastID
db.run(sql, params, (err) => {
  console.log(this.lastID); // undefined
});
```

```
// ✅ Regular function — correct "this" binding
db.run(sql, params, function(err) {
  console.log(this.lastID); // Works: e.g., 5
});
```

In short

- When we don't use `this`, prefer an arrow function for simplicity.
- When we use `this`, prefer a regular function, because an arrow function has no `this` of its own (it uses the `this` of the surrounding scope).